

Product Requirements Document

RELASI

Village Tourism Website

+

KONEKSI

Integrated Content Management System (Sanity)

Field	Value
Program	KKN Mekar Banjar — Community Service Program
Document purpose	Master rulebook for AI coding assistants (Cursor / Antigravity). All instructions herein are binding. Where this document is silent, the AI developer MUST ask the human team before making assumptions.
Version	1.0 (MVP scope)
Date	2 July 2026
MVP deadline	Before end of July 2026 (hard deadline)
Status	Approved for development

How the AI Developer Must Use This Document

- This PRD is the **single source of truth**. Do not invent features, pages, fields, or libraries that are not specified here.
- Boxes marked **■ STRICT RULE** are non-negotiable constraints. Violating them is considered a failed implementation.
- If a requirement is ambiguous or missing, **stop and ask the human team**. Never guess.
- Scope discipline: anything labelled “Out of scope” must not be built, scaffolded, or stubbed, even partially.

1. Project Overview & Goals

1.1 Background

RELASI is a tourism promotion website for the villages served by the KKN Mekar Banjar community service program. It introduces village tourist attractions, homestays, and local nature (flora & fauna) to domestic and international visitors, so they have a clear picture of the village before arriving.

KONEKSI is the integrated content management layer, built on Sanity (headless CMS). It lets local villagers publish news, updates, and articles through a clean WYSIWYG editor **without reading or writing any code**.

The project starts from scratch. The previous team's website failed because it was poorly structured, slow (heavy unoptimized images), and built without planning. This PRD exists specifically to prevent a repeat of those failures.

1.2 Goals

#	Goal	How this PRD enforces it
G1	Fast, lightweight website on any connection (rural mobile networks included).	Mandatory next/image usage, Lighthouse ≥ 90 mobile target (Section 7).
G2	Villagers publish content independently, zero code.	KONEKSI CMS scoped to article publishing only (Sections 4.3, 6).
G3	Reach both Indonesian and international audiences.	Bilingual ID/EN with subpath routing (Section 4.1).
G4	Clean handover to the next KKN team after 1 year.	Shared-email account ownership and handover protocol (Section 7).

1.3 Non-Goals (Out of Scope for MVP)

- No internal booking / payment / reservation system. Homestay contact is a 1-click external redirect (WhatsApp) only.
- No user accounts, login, comments, or membership on the public website.
- No villager CMS access beyond articles (no homestay editing, no page editing, no file management).
- No self-hosted video. All video is embedded from YouTube.
- No native mobile app. Responsive web only.

2. Target Audience

User type	Who they are	Primary needs	Device / context
Tourist — domestic (website viewer)	Indonesian travelers researching the village before visiting.	Attraction info, homestay list with instant WhatsApp contact, articles/news, downloadable guidebook.	Mostly mobile, variable connection speed. Default language: Indonesian.
Tourist — international (website viewer)	Foreign travelers discovering the village online.	Same as above, in English. Clear visuals compensating for language gaps.	Mobile + desktop. Switches to English via language switcher.
Villager (KONEKSI CMS user)	Local residents with no coding skills, possibly limited digital literacy.	Write and publish articles: text, images, YouTube embeds — through a simple WYSIWYG editor only.	Shared computers or phones. Uses Sanity Studio; never touches code or hosting.
KKN team / next team (maintainer)	Students who deploy, maintain, and eventually hand over the project.	Clean codebase, documented setup, easy content of hardcoded sections via code files.	Developer environment (VS Code / Cursor).

3. Tech Stack & Architecture Rules

3.1 Approved Stack (exhaustive — do not add without approval)

Layer	Technology	Notes / constraints
Framework	Next.js (App Router, latest stable)	TypeScript required. App Router only — do not use Pages Router.
Styling	Tailwind CSS	No other CSS frameworks. No styled-components, no CSS-in-JS libraries.
CMS	Sanity (headless) — branded "KONEKSI"	Sanity Studio deployed for villager access. Free tier.
Internationalization	next-intl	Subpath routing /id and /en. See Section 4.1.
Rich text rendering	@portabletext/react	Official Portable Text renderer. See STRICT RULE in Section 4.3.
Video	YouTube embeds only	Use lazy-loaded embeds (e.g., lite-youtube-embed pattern or next/dynamic) to avoid loading the YouTube player on initial page load.
Flip-book	react-pageflip (or equivalent lightweight page-flip library)	For the flora & fauna booklet. Content is hardcoded (Section 4.5).
Hosting	Vercel (free/hobby tier)	Automatic deploys from Git main branch.
Dev tools	Cursor / Antigravity (AI assistants)	This PRD is their master rulebook.

3.2 Architecture Rules

- **Rendering strategy:** Static Site Generation (SSG) with Incremental Static Regeneration (ISR) for article pages (revalidate ≤ 60 s or on-demand via Sanity webhook). Do not use client-side data fetching for primary page content.
- **Content split:** Article content lives in Sanity. Homestays, flora/fauna booklet, attractions, and downloads are **hardcoded** in the repository as typed data files (TypeScript objects or JSON). Do not create Sanity schemas for them.
- **Folder structure:** conventional Next.js App Router layout — `app/[locale]/...` for routes, `components/`, `lib/` (Sanity client, queries), `data/` (hardcoded content), `messages/` (translation JSON for next-intl).
- **Environment variables:** Sanity project ID, dataset, and API version go in `.env.local`; never hardcode credentials.
- **Dependencies:** keep minimal. Every new npm package beyond Section 3.1 requires human approval.

■ STRICT RULE — NEXT.JS IMAGE OPTIMIZATION (mandatory, zero exceptions)

1. Every image on the public website **MUST** be rendered through the `next/image` `<Image>` component. Plain `<img>` tags are forbidden anywhere in the frontend.
2. Every `<Image>` **MUST** define width/height (or `fill` with a sized container) to prevent layout shift, and a meaningful `alt` text.
3. Above-the-fold hero images use `priority`; all other images lazy-load (default behavior — do not disable it).
4. Provide sizes on responsive images so mobile devices never download desktop-sized files.
5. Sanity images **MUST** be served through the Sanity image CDN URL builder (`@sanity/image-url`) combined with `next/image`, so they are resized and converted (WebP/AVIF) automatically.
6. Local/static images (homestays, booklet, attractions) **MUST** be imported statically so Next.js can optimize them at build time.
7. Rationale: the previous website died because of heavy unoptimized images. Any code that bypasses image optimization is an automatic implementation failure.

4. Feature Breakdown & Requirements

4.1 Bilingual Support (Indonesian / English)

- Routing: subpath-based — `/id/...` and `/en/...` — implemented with **next-intl**.
- Default locale: **Indonesian (id)**. The root `/` serves/redirects to Indonesian. **No automatic browser-language detection**.
- A visible language switcher (ID | EN) sits in the top navigation on every page. Switching preserves the current page where a translation exists.
- All UI strings (nav, buttons, labels, footer) live in translation JSON files (`messages/id.json`, `messages/en.json`). No hardcoded UI strings in components.
- Articles: bilingual at the field level (title/body per language, Section 6). If an article's English content is empty, the English site shows the Indonesian content with a small “Available in Indonesian only” note — never a broken/blank page.
- SEO: each page emits correct `hreflang` alternates and locale-aware metadata.

4.2 Performance

- Target: Lighthouse Performance ≥ 90 on **mobile** for Home, one article page, and the Homestay page.
- Follow the STRICT RULE image box in Section 3.2 — this is the core of performance.
- YouTube embeds must be lazy/facade-loaded: render a thumbnail + play button first; load the `iframe` player only on click.
- Fonts: use `next/font` with self-hosted Google fonts (max 2 font families). No render-blocking font links.
- No heavy client libraries for content pages. Client components only where interactivity requires them (language switcher, flip-book, mobile menu).

4.3 KONEKSI — Content Management (Sanity)

- Villagers access Sanity Studio (branded “KONEKSI”, Indonesian labels where Sanity allows) to create and publish **articles only**.
- Editor experience: title, cover image upload, category selection, and a rich-text body supporting headings, bold/italic, lists, links, inline images with captions, and a YouTube-embed block (paste URL).
- Publishing flow: villager writes → clicks Publish → site updates automatically via ISR/webhook within ≤ 60 seconds. No developer involvement.

■ STRICT RULE — VILLAGER CMS SCOPE (hard permission boundary)

Villagers can ONLY create/edit/publish documents of type `article` (and select from existing category values).

Villagers CANNOT edit homestays, attractions, the flora/fauna booklet, downloads, site settings, or any page structure — those do not exist in Sanity at all (they are hardcoded in the repository).

Do not create any additional Sanity document types beyond Section 6 without human approval.

■ STRICT RULE — SANITY PORTABLE TEXT RENDERING (mandatory)

1. Article bodies are stored as Sanity **Portable Text**, NOT HTML and NOT Markdown. Never treat the body as a string.
2. The frontend MUST render bodies with `@portabletext/react` (`<PortableText value={body} components={...} />`) and a custom components map.
3. The components map MUST implement: block styles (`h2`, `h3`, `normal`, `blockquote`), marks (`strong`, `em`, `link` — external links get `rel="noopener noreferrer"` and `target="_blank"`), lists (`bullet`, `number`), an `image` type rendered through `next/image` + `@sanity/image-url` (with caption/alt), and a `youtube` type rendered as a lazy embed.
4. Unknown block types must render nothing (fail silently with a console warning in dev), never crash the page.
5. Never use `dangerouslySetInnerHTML` for article content.

4.4 Homestay Directory

- Data source: **hardcoded** typed array in `data/homestays.ts`. No Sanity schema, no admin UI.
- Each homestay entry: name, short description (ID + EN), 1–5 photos (static imports), price range text, facilities list, and a WhatsApp number.
- Card layout in a responsive grid; each card has one primary button: “Contact via WhatsApp”.
- The button is a 1-click external redirect to `https://wa.me/<number>?text=<prefilled greeting>`, opening in a new tab. Prefilled greeting is locale-aware (ID or EN template).
- **No internal booking system**: no forms, no availability calendar, no database.

4.5 Interactive Flora & Fauna Booklet

- Presentation: interactive flip-book with page-turn animation (`react-pageflip` or equivalent).
- Content: **hardcoded**; booklet pages/species data provided by the KKN team as static assets in the repository (`data/booklet.ts` + images). Villagers cannot edit it.
- Each page/species: photo, common name (ID + EN), scientific name, short description (ID + EN).
- Must be usable on mobile: swipe/tap to turn pages; graceful fallback to a simple sequential viewer if the flip library fails to load.
- All booklet images go through `next/image` static imports (see STRICT RULE 3.2).

4.6 Public File Showcase (KKN Output)

- **After-movie**: embedded YouTube video (lazy facade embed), with a short bilingual description.
- **Digital guidebook**: a static PDF file stored in the repository (`public/files/guidebook.pdf`), presented with a preview cover image and a Download button. Updating it requires a code commit + redeploy — this is intentional; it is not CMS-managed.
- Section may list additional KKN outputs later using the same hardcoded pattern.

4.7 General UI Requirements

- Fully responsive (mobile-first). Test at 360px, 768px, 1280px widths.
- Sticky top navigation with logo, 7 section links (Section 5), and the language switcher; collapses to a hamburger menu on mobile.
- Footer: village address, social links, KKN Mekar Banjar credit, and quick links.
- Accessibility: semantic HTML, alt text on all images, visible focus states, sufficient color contrast (WCAG AA).

5. Information Architecture (Sitemap)

Seven top-level sections, identical structure for both locales. Routes shown for `/id`; mirror them under `/en` (slugs of static pages are translated in the nav label only; URL path segments stay in English for simplicity and consistency).

#	Section (nav label ID / EN)	Route	Content source
1	Beranda / Home	<code>/id</code>	Hero, highlights of attractions, latest 3 articles, homestay teaser, after-movie teaser.
2	Wisata / Attractions	<code>/id/attractions</code>	Hardcoded attraction list (<code>data/attractions.ts</code>): name, photos, description ID+EN, location/map link. Detail pages at <code>/attractions/[slug]</code> .
3	Homestay / Homestay	<code>/id/homestay</code>	Hardcoded (Section 4.4).
4	Artikel / Articles	<code>/id/articles</code>	Sanity. Listing with cover, title, excerpt, date, category filter. Detail at <code>/articles/[slug]</code> (Portable Text).
5	Buklet Flora & Fauna / Flora & Fauna Booklet	<code>/id/booklet</code>	Hardcoded flip-book (Section 4.5).
6	Galeri & Unduhan / Gallery & Downloads	<code>/id/downloads</code>	After-movie embed + guidebook download (Section 4.6).
7	Tentang / About & Contact	<code>/id/about</code>	Village profile, KKN program info, contact details, embedded map link.

- Additionally: 404 page (bilingual), `sitemap.xml`, `robots.txt`.
- Do not create any route not listed here without human approval.

6. Data Models (Sanity Schemas)

Exactly **two** Sanity document types exist. Nothing else.

6.1 Document: `article`

Field	Type	Required	Notes
<code>title_id</code>	string	Yes	Indonesian title.
<code>title_en</code>	string	No	English title. Empty → fallback rule in Section 4.1.
<code>slug</code>	slug (from <code>title_id</code>)	Yes	Unique, auto-generated, editable.
<code>coverImage</code>	image (with alt string field)	Yes	Rendered via Sanity CDN + <code>next/image</code> .
<code>category</code>	reference → category	Yes	Single category per article.
<code>excerpt_id</code>	text (max ~200 chars)	Yes	Listing preview, Indonesian.
<code>excerpt_en</code>	text (max ~200 chars)	No	Listing preview, English.
<code>body_id</code>	Portable Text (blocks + image + youtube)	Yes	Blocks: normal, h2, h3, blockquote; marks: strong, em, link; lists; custom image object (asset + alt + caption); custom youtube object (url string, validated).
<code>body_en</code>	Portable Text (same config)	No	English body.
<code>publishedAt</code>	datetime	Yes	Defaults to now.
<code>authorName</code>	string	Yes	Free text. No author accounts/documents.

6.2 Document: category

Field	Type	Required	Notes
title_id	string	Yes	e.g., Berita, Acara, Cerita Desa.
title_en	string	Yes	e.g., News, Events, Village Stories.
slug	slug	Yes	Used for filtering on the articles page.

- Categories are seeded by the KKN team. Villagers select existing categories; creating new ones is allowed but keep the Studio UI simple.

6.3 Hardcoded Data Files (repository, TypeScript)

File	Contents
data/homestays.ts	Homestay entries per Section 4.4 (typed interface).
data/attractions.ts	Attraction entries per Section 5 row 2.
data/booklet.ts	Flora/fauna booklet pages per Section 4.5.
data/downloads.ts	After-movie YouTube URL + guidebook file metadata.
messages/id.json, messages/en.json	All UI translation strings (next-intl).

- Each hardcoded file must export a TypeScript interface so the AI assistant and future maintainers get type errors on malformed entries.

7. Success Metrics & Handover Protocol

7.1 Success Metrics (MVP acceptance)

Metric	Target	How verified
Lighthouse Performance (mobile)	≥ 90 on Home, one article detail, Homestay page	Lighthouse in Chrome DevTools / PageSpeed Insights on production URL.
Image discipline	0 plain tags; all images via next/image	Codebase grep + visual audit before launch.
Bilingual completeness	100% of UI strings translated; no missing-key warnings	next-intl runtime check + manual walkthrough of both locales.
CMS usability	A villager publishes a test article (text + image + YouTube) end-to-end without developer help	Live onboarding session before handover.
Publish latency	Article visible on site ≤ 60s after Publish	Timed test with ISR/webhook.
Deadline	MVP live on production domain before end of July 2026	Vercel production deployment.
Post-handover adoption (soft)	≥ 1 villager article per month	Informal tracking by next team; not enforced.
Uptime	Vercel platform default	No extra monitoring tooling for MVP.

7.2 Account Ownership & Handover Protocol

- **Single shared email** (already held by the KKN team) owns ALL accounts from day one: Vercel, domain registrar, Sanity, and the Git repository/organization. No account may be registered under a personal team-member email.

- Hosting and domain costs are covered for **1 year**. Before expiry, the next KKN team (or village administration) renews using the same accounts — credentials are handed over as-is; **no re-registration or ownership transfer is required**.
- Handover package: (1) shared-email credentials + 2FA recovery codes, (2) repository access, (3) a README covering local setup, deploy flow, how to edit each hardcoded data file, and how to invite/remove Sanity users, (4) an informal training session for the receiving team and villager editors (no formal SOP document required).
- Villager Sanity accounts: invited as members with the most restricted practical role; the shared email retains admin.

7.3 Definition of Done (MVP checklist)

- All 7 sections of Section 5 implemented in both locales.
- KONEKSI Studio deployed, article + category schemas live, villager test-publish passed.
- Both STRICT RULE boxes (image optimization, Portable Text, CMS scope) verified.
- Success metrics table 7.1 fully passed.
- Handover README written and stored in the repository root.

End of document. Where this PRD is silent, ask the human team — do not assume.